SC24: International Conference for High Performance Computing, Networking, Storage and Analysis | 979-8-3503-5291-7/24/$31.00 ©2024 IEEE | DOI: 10.1109/SC41406.2024.00073

LEXIQL: Quantum Natural Language Processing on NISQ-era Machines

Daniel Silver
Northestern University
silver.da@northeastern.edu

Aditya Ranjan
Northestern University
ranjan.ad@northeastern.edu

Rakesh Achutha
IIT-BHU, Varanasi
achutha.rakesh.mat20@iitbhu.ac.in

Tirthak Patel
Rice University
tirthak.patel@rice.edu

Devesh Tiwari
Northestern University
d.tiwari@northeastern.edu

***Abstract*—The rapid evolution of quantum hardware is propelling quantum computing to new frontiers. Nonetheless, the potential of natural language processing in the quantum paradigm (QNLP) is yet to be explored, including for Noisy Intermediate-Scale Quantum (NISQ) machines. To explore the QNLP frontier, we introduce LEXIQL, a novel noise-aware QNLP technique for text classification on NISQ quantum machines. LEXIQL employs an incremental data injection approach to process textual data in a quantum circuit. It also develops new and effective training methods, such as leveraging a diverse mix of expressible and shallow quantum circuits for the QNLP task of text classification. Our extensive evaluation using Yelp, IMDB, and Amazon datasets (along with synthetic QLNP datasets) demonstrates the effectiveness of LEXIQL's noise-aware design in both ideal and noisy environments.**

***Index Terms*—Quantum Computing, Quantum Machine Learning, Natural Language Processing**

I. INTRODUCTION

Natural Language Processing (NLP), an interdisciplinary domain combining computer science with linguistics, has undergone rapid advancement, creating transformative tools like ChatGPT [46]. This evolution is reshaping societal interactions by enabling real-time language translation, developing tools for sentiment analysis that capture human emotions, and making specialized medical information more accessible [41], [61], [63]. Running training and inference tasks for advanced classical NLP models require extensive computational resources [21], [13]. On the other hand, recent research has demonstrated that Quantum Machine Learning (QML) offers significant promise, with the potential of outperforming traditional machine learning capabilities, accelerating many high-performance computing (HPC) tasks [50], [34], [23], [32]. Given the advancements in NLP and the emerging potential of QML, Quantum Natural Language Processing (QNLP) holds the potential, though not theoretically demonstrated yet, to surpass the challenges of conventional NLP methods [69].

Unfortunately, the application of QML for natural language processing (NLP) has not been demonstrated beyond the scale of small, synthetic datasets. A key foundational NLP task, text classification, has been investigated in prior works [2], [26]. However, these prior works have faced challenges in performing text classification on non-synthetic datasets and instead are only effective on small, artificial data (our motivation section provides an in-depth understanding of the current state of the art and key limitations) and are not designed for current erroneous NISQ-era machines.

The work in [26] faces practical limitations, having large resource requirements, needing at least one qubit (quantum bit) per word – which is particularly prohibitive as the noise on current quantum computers increases with the number of qubits – and requiring a method to parse grammar that is compatible with real-world datasets [60]. The work by [2] faces a different set of challenges, as its technique of averaging all word data together significantly hinders the capability of data classification, leading to poor classification accuracy for datasets with more than a few words. *Motivated by the identified opportunity gaps in QNLP, our goal is to develop an effective quantum text classifier.*

Contributions of LEXIQL. LEXIQL is a novel design and implementation for text classification tasks on quantum computing machines. Amidst the limited but emerging research landscape in Quantum Natural Language Processing (QNLP), LEXIQL represents a notable advancement, exhibiting predictive capability on realistic datasets under noisy conditions of Noisy Intermediate-Scale Quantum (NISQ) machines.

Key Design Elements. I. LEXIQL *introduces a novel incremental data injection approach to process textual data in a quantum circuit.* LEXIQL's approach reduces the information loss suffered by prior approaches, which naively rely on averaging all the data upfront in one shot. However, introducing data incrementally interferes with the state of the quantum circuit and poses a new challenge. LEXIQL proposes a novel concept of EIP (Encode-Inject-Process) Cells to mitigate it.

SC24, November 17-22, 2024, Atlanta, Georgia, USA
979-8-3503-5291-7/24/$31.00 ©2024 IEEE
Authorized licensed use limited to: Universita degli Studi di Salerno. Downloaded on July 02,2026 at 10:13:19 UTC from IEEE Xplore. Restrictions apply.

II. LEXIQL *designs EIP (Encode-Inject-Process) cells that compress prior state information, inject new data for learning, and use a processing layer to train for text classification.* LEXIQL's EIP cells use a quantum encoder to compress data and free up qubits to inject new data. This architecture enables LEXIQL to train effectively and mitigate the state-data interference challenges created by incremental data injection.

III. Unfortunately, a single learner with EIP cells is not effective in the presence of noise on NISQ-era machines. Therefore, LEXIQL *augments its EIP cells with multiple learners, forming a diverse quantum ensemble.* LEXIQL's diverse ensemble approach enhances accuracy and noise resilience by combining multiple quantum models – and by carefully navigating the trade-off between error-proneness (answer fidelity) and expressibility (learning capability) of individual learner models. LEXIQL *creates a novel diverse mix of learner models: highly-expressible (higher learning capability) but deep circuits (more noise-resilient) and low-depth (more noise-resilient), but less expressible (lower learning capability) quantum circuits.* LEXIQL employs multiple-axes projection to improve accuracy without requiring re-training in different dimensions.

Evaluation Results. Our extensive evaluation of five datasets, comprising synthetic and real data, demonstrates the effectiveness of LEXIQL, in both ideal and noisy environments. LEXIQL achieves 87% average accuracy over five datasets in ideal simulation. It is also evaluated under different noise scenarios and achieves a 79% average accuracy under IBM's Hanoi quantum computer's noise environment. LEXIQL is available at https://zenodo.org/records/10980744.

II. BACKGROUND AND MOTIVATION

Foundations of Quantum Computing. Qubits are the fundamental units of quantum computing, whose state can be expressed as a superposition of basis states $|0\rangle$ and $|1\rangle$, and collapses to a definite basis state upon measurement. Mathematically, a qubit state $|\ \rangle$ is expressed as $|\ \rangle = \alpha\,|0\rangle + \beta\,|1\rangle$, where $|\alpha|^2$ and $|\beta|^2$ are the probabilities of observing states $|0\rangle$ and $|1\rangle$, respectively [44]. We represent a single qubit state using a Bloch sphere. A Bloch sphere is a unit sphere where the positive and negative z-axis represent the $|0\rangle$ and $|1\rangle$ states, respectively. Quantum gates are unitary operations that affect the state of the quantum system. Typically, quantum circuits, a series of quantum gates, are measured in a particular measurement basis (e.g., X, Y, or Z), known as a projection.

Variational quantum circuits (VQCs) are quantum circuits that optimize parameters θ, where θ represents the set of p tunable parameters in the circuit, denoted as $\gamma_1\ \gamma_2\ \ \gamma_p$. The fixed gates, denoted as x, define the circuit's structure used to create quantum entanglement. Represented as $U(\theta\ x)$, VQCs, akin to classical neural networks, employ gradient-based optimizers to iteratively update parameters. The optimization process, involving an objective function O and input Z under constraints C, aims to find optimal parameters θ^* [15].

$$\theta^* = \arg\min_{\theta \in C} O(U(\theta\ x)\ Z) \tag{1}$$

Noisy Intermediate-Scale Quantum (NISQ) devices, the current slate of quantum computers, are vulnerable to errors. These errors arise from environmental factors, imprecise control, and decoherence [11]. As quantum circuits require a series of gates, errors can cascade and create a significant loss in circuit fidelity. Therefore, it is important to employ noise-aware techniques to make the best use of NISQ devices.

Natural Language Processing (NLP) and Quantum NLP (QNLP). NLP addresses a wide range of challenges, including classification, text generation, translation, and sentiment analysis – analyzing and studying opinions and emotions in text [31], [25], [45]. Classification is foundational to NLP [24], [30], being fundamental to applications such as text generation, where the choice for each generated word must be predicted, and to spam detection [12], [22].

Text classification is foundational for more complex tasks such as text generation and spam detection [62], [22]. Indeed, the classical NLP evolved to more complex tasks, building upon the text classification tasks such as sentiment analysis [24], [40], [62]. Therefore, as a first natural step in the regime of Quantum NLP, we focus on designing an effective text classifier for sentiment analysis.

In the realm of Quantum NLP, two primary approaches are prominent for tackling text classification-related NLP tasks. The first approach, Categorical Compositional Distributional (commonly referred to as DisCoCat) [10], stems from a mathematical analogy between quantum physics and linguistics, employing category theory as its foundation. DisCoCat is built by considering word types (e.g., noun, verb) in a grammar-based approach using a tensor representation. DisCoCat involves projecting tensors to a common space without requiring a full representation, giving it an advantage over previous tensor-based methods [17]. While DisCoCat has convenient theoretical formalisms, it poses restrictions — requiring a specific grammatical structure that must be comprehensible with its syntax parser. DisCoCat's strict syntax requirements and size requirements are challenging for the current capabilities of quantum circuits, especially in the presence of errors [60]. In this work, we utilize the implementation of DisCoCat provided by Lambeq for our experiments, denoted as DISCOCAT [26].

Another QNLP approach, referred to as AMPMEAN, is based on processing words without explicitly modeling grammar, being more akin to traditional classical deep-learning models [2]. This approach is shown in Figure 1, whereby words are first converted to word vectors before being placed on a quantum circuit for optimization. AMPMEAN has resource advantages over DisCoCat, being able to process a large sequence of words without requiring additional qubits. However, as each word in AMPMEAN is averaged together, there may be a significant loss in information, limiting the total amount of available data and hindering classification.

Limitations of Existing QNLP Techniques. DisCoCat models face practical limitations, requiring a minimum of one qubit for each word in a text, which poses a significant constraint. As the number of qubits increases, quantum systems expand

Authorized licensed use limited to: Universita degli Studi di Salerno. Downloaded on July 02,2026 at 10:13:19 UTC from IEEE Xplore. Restrictions apply.

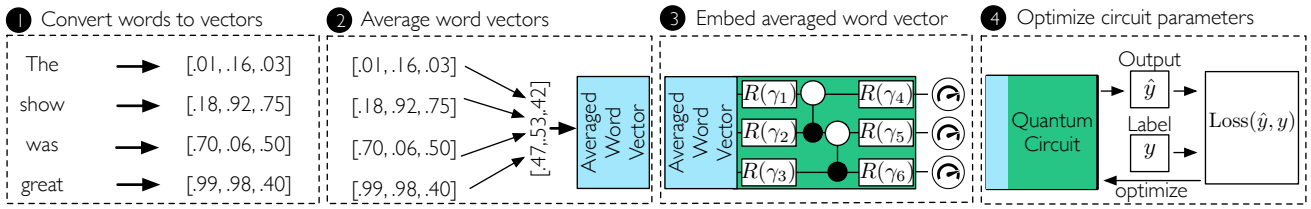


Fig. 1. AMPMEAN *approach to quantum natural language processing [2] procedure shown. Words are converted to vectors and then averaged to maintain a fixed circuit size given variable length data before classification. The* $R(\gamma_i)$ *are one-qubit gates with tunable parameters; the gates connecting two qubits (two horizontal lines) represent entangling CNOT gates; the operations at the end of the circuits (circular meters) represent measurements.*

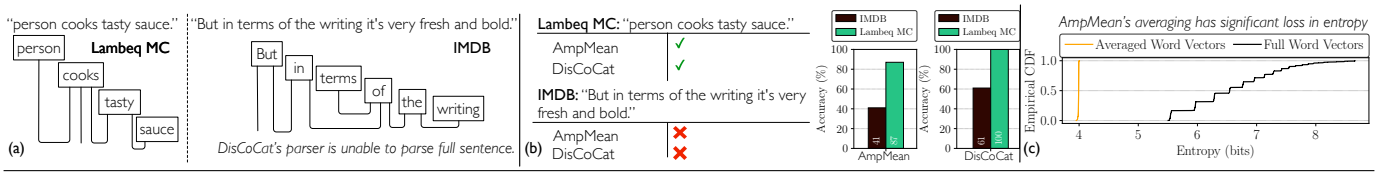


Fig. 2. *(a) DisCoCat syntax parsing applied to two sentences [26]. (b)* AMPMEAN *and* DISCOCAT *[2], [26] accuracy shown for realistic (IMDB) and synthetic (LambeqMC) datasets. (c) Entropy of data shown using* AMPMEAN*'s averaging approach when compared to original word vectors.*

in size exponentially, rendering the simulation of large qubit counts on classical computers – a common practice for training traditional Quantum Machine Learning (QML) models [20], [56] – impractical. Furthermore, quantum machines in the NISQ era typically possess tens of qubits, which restricts the feasibility of implementing these models, especially on NISQ devices where gate operations are often error-prone [39].

Moreover, the reliance on parsers that may inaccurately analyze syntax further restricts DisCoCat's effectiveness. We demonstrate an example of an imperfect parser impacting the processing of a sentence in Figure 2 (a), where a more realistic sentence from IMDB is unable to be parsed completely due to complex structures, leading to an imperfect grammatical representation. Our findings likewise find that although DisCoCat models excel with small, synthetic datasets, their performance significantly drops when applied to more complex, real-world data, such as IMDB movie reviews. As Figure 2 (b) shows, DisCoCat achieves high accuracy on the synthetic LambeqMC data, but its accuracy drops to 61% on the real-world IMDB data, motivating the need for a solution for complex datasets.

Similarly, AMPMEAN faces its own set of challenges, particularly with its method of condensing sequential data into a fixed-length vector through averaging. While this approach is effective for quantum text classification in shorter documents (e.g., LambeqMC), it suffers in real-world datasets. Our analysis, demonstrated in Figure 2 (b), reveals that this data reduction significantly impairs the model's ability to perform text classification effectively on IMDB data (41% accuracy). Figure 2 (c) investigates the underlying causes of this. To provide context, entropy is a measure of the unpredictability or randomness in a dataset, reflecting the amount of information it contains. In our analysis, we contrast the entropy levels when data is averaged against when it remains whole, aiming to investigate the information loss that occurs. We observe that information loss happens in all cases due to averaging, with some scenarios showing a significant decrease in entropy by over 50%. This information loss clarifies why AMPMEAN underperforms when dealing with realistic datasets, as the averaging process significantly dilutes the complexity of the data. These issues tend to be further exacerbated under noise present in NISQ-era quantum machines.

To address these challenges, we propose LEXIQL, a solution for high-quality quantum text classification that introduces multiple novel techniques that work effectively for more complex datasets and under noisy quantum conditions.

III. LEXIQL: DESIGN

A. *Overview of* LEXIQL*'s Design.*

LEXIQL introduces a novel design for text classification, as summarized in Figure 3. It first performs incremental data injection, a method that deviates from the traditional approach of initial data injection, where all classical data is injected at the beginning of the circuit, enhancing sequential data processing [9], [64], [65]. LEXIQL's approach, then, creates interference between the state of the circuit and the new data, necessitating a strategy to clear space for new data. LEXIQL addresses this through the use of a novel Encode-Inject-Process (EIP) cell architecture designed to manage sequential data and reduce interference between the quantum state and data. Further, LEXIQL's diverse ensemble approach enhances accuracy and noise resilience by combining multiple quantum models based on varied circuit characteristics. Finally, LEXIQL proposes multiple-axes projection, a technique to enable richer information extraction from quantum circuits. Next, we discuss these steps in detail.

B. *Key Idea 1:* LEXIQL*'s Incremental Data Injection.*

Unlike the traditional approach where all data is injected at the beginning of the circuit as seen in previous works [9], [64], [65], LEXIQL adopts an incremental data injection strategy. Prior methods achieve data injection by averaging

Authorized licensed use limited to: Universita degli Studi di Salerno. Downloaded on July 02,2026 at 10:13:19 UTC from IEEE Xplore. Restrictions apply.

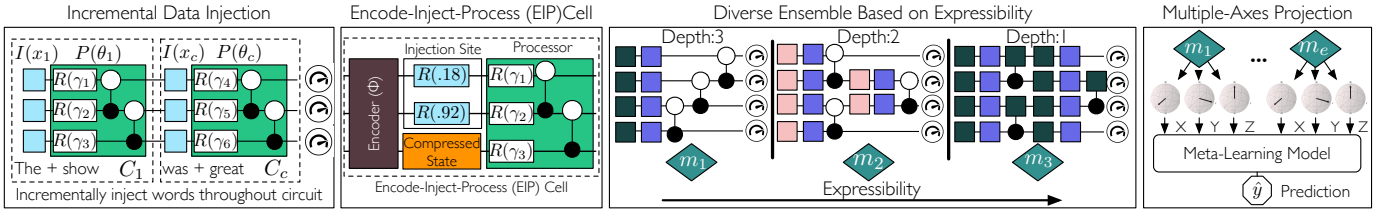


Fig. 3. *Key design elements of* LEXIQL*: (1) Incremental data injection, (2) EIP cells, (3) Ensemble of diverse learners, and (4) Multiple-axes projections.*

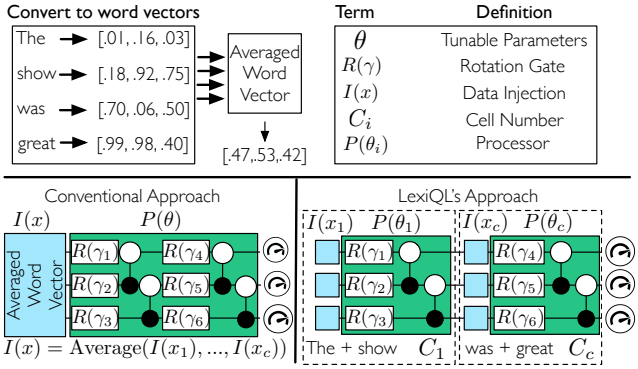


Fig. 4. *State-of-the-art approach to processing sequential data on a quantum circuit (left) [2] and* LEXIQL*'s incremental data injection approach (right).*

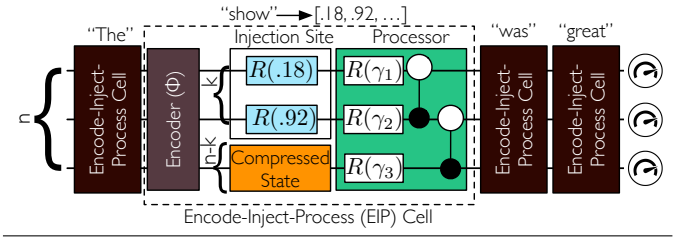


Fig. 5. LEXIQL*'s Encode-Inject-Process (EIP) cells applied in a sequence perform text-classification. Each word within a document has a corresponding EIP Cell consisting of a trained encoder, an injection site, and a processor.*

word vectors [2], utilizing amplitude embedding, a quantum technique capable of encoding 2^n classical data elements onto n qubits. Averaging the data is required to fit variable-length data in a fixed size so that amplitude embedding can be used. This prior conventional approach leads to significant information loss due to the averaging of all document words into a single vector. In contrast, LEXIQL processes words individually throughout the circuit, avoiding the information loss associated with averaging, as shown in Figure 4. LEXIQL's method employs incremental injections of data x at specific sites $(I(x_i))$ and utilizes tunable parameters (θ_i) in processing modules $(P(\theta_i))$, forming a cell-based architecture. Formally, our ansatz, $U(\theta\ x)$, takes a multiple-cell form to solve for optimal parameters for text classification θ^*:

$$U(\theta\ x) = \prod_{i=1}^{c} P(\theta_{c+1-i}) I(x_{c+1-i}) \tag{2}$$

Here, i denotes the cell number, and c, determined by the document's word count, is the total number of cells.

While promising (as also shown in Sec. V), incremental data injection introduces a new challenge: state-data interference. The state-data interference occurs because newly injected data can disrupt pre-existing quantum states. LEXIQL addresses this by utilizing a cell-based architecture with EIP (Encode-Inject-Process) cells to reduce interference, thus improving its data injection strategy. The design of LEXIQL's EIP cell architecture is detailed in the following section.

C. Key Idea 2: LEXIQL's Encode-Inject-Process (EIP) Cell.

As shown in Figure 4, the architecture of LEXIQL consists of repeated cells of data injection I to inject data incrementally and processor P modules to optimize the text classification task. However, data injected incrementally on qubits containing processed states interferes with the text classification task, creating state-data interference. For example, in Figure 4 with LEXIQL's approach, the injection of the word "was" occurs on the same qubits as the processed C_1 cell, interfering with the ability to perform classification by overriding the qubits where classification data has been processed.

To overcome this challenge, LEXIQL incorporates an encoder in each layer for data compression. The encoder allows for compression of information stored in n qubits to n-k qubits. This allows k qubits to become available as an injection site for new data w. Repeated serially, the encoder enables compatibility with arbitrarily long documents consisting, requiring only a fixed number of qubits, forming an EIP-cell architecture as shown in Figure 5.

1) Quantum Encoder.: LEXIQL leverages a quantum autoencoder, a quantum circuit designed for encoding data into a lower-dimensional space and subsequently decoding it from this compressed representation, as a fitting choice for mitigating state-data interference and enhancing classification [19], [43]. This approach, being capable of efficiently compressing complex quantum data, aligns with the requirements for repeatedly compressing quantum states to mitigate the adverse impact of state-data interference.

Specifically, we define the encoder, sub-component of the autoencoder as $E(\phi)$, where $E(\phi)$ represents a collection of unitary operations acting on the entire input space spanning n qubits. The goal of the encoding process is to compress n

Authorized licensed use limited to: Universita degli Studi di Salerno. Downloaded on July 02,2026 at 10:13:19 UTC from IEEE Xplore. Restrictions apply.

qubits of information (containing 2^n numbers) onto a reduced space comprising $n-k$ qubits (containing 2^{n-k} numbers). Within this context, we define "compressed state" as the space corresponding to $n-k$ qubits, while "injection site" corresponds to the remaining k qubits.

The encoder has two distinct outputs: the first is the compressed data across $n-k$ qubits, and the second, ideally data-free, spans k qubits. Although the encoder reduces state-data interference, its repeated use introduces potential errors due to additional gates. Thus, optimizing the encoder's design to minimize gate errors is crucial for maintaining LEXIQL's efficiency when NISQ-era noise is present.

2) Encoder Ansatz.: LEXIQL's encoder ansatz is designed to minimize the impact of NISQ-era noise, addressing the challenge of error accumulation due to the repeated use of gates within the encoder. To enhance fidelity, the encoder features a straightforward design, prioritizing efficiency with a limited number of CNOT gates for essential entanglement and incorporating a single tunable rotation gate per qubit in each layer. However, applying CNOT gates between non-adjacent qubits is not possible directly without the use of SWAP operations.

SWAP operations (SWAPs) facilitate computation among non-adjacent qubits in quantum circuits by iteratively repositioning qubits for direct interaction. LEXIQL strategically reduces the use of SWAPs to adapt to the physical topology of qubits. The physical topology of qubits refers to the spatial configuration and connectivity of qubits within a quantum computer, determining which qubits can directly interact with each other. This approach involves optimizing the placement of CX gates to avoid excessive SWAPs, especially in configurations like a linear qubit array where distant qubit interactions are costly. By reconfiguring entanglement patterns to suit the machine's topology, LEXIQL effectively lowers the reliance on SWAP operations, integrating changes into the repeated Encode-Inject-Process cell structure for improved performance on a noisy quantum computer.

LEXIQL's encoder functions by compressing n qubits of information onto $n-k$ qubits, followed by angle embedding features on the freed-up k qubits. Utilizing angle embedding functions to rotate a single qubit by a certain angle on a given axis; $2k$ features of a word are injected on the freed-up qubits. Features $[0\ k]$ are injected using rotations along the X-axis, followed by features $[k\ 2k]$ injected as rotations along the Y-axis on the k qubits. After data injection, the processor module processes the previous state and inputs data to create a new state (which is a better representative of the sentiment). Formally, the extended ansatz structure of c EIP cells takes the form of Eq. 3 with the encoder parameters ϕ.

$$U(\theta\ x\ \phi) = \prod_{i=1}^{c} P(\theta_{c+1-i}) I(x_{c+1-i}) E(\phi) \tag{3}$$

Figure 5 visually shows the EIP cell in an example sentence where each word has an associated EIP cell with a respective encoder, injection site, and processor. Next, we discuss the training procedure of LEXIQL's EIP cell modules.

Algorithm 1 Training Procedure of LEXIQL's Encoder

Require: Documents D, Number of iterations N, Encoder $E(\phi)$, Compressed State $n-k$, Injection Site Qubits k, Pure State s, Auxiliary Qubit a

for i = 1 to N **do**
  Generate batch of data $x_i \leftarrow D$
  Select word from x_i at random $w \leftarrow x_i$
  Inject word $\leftarrow RX(w)\ RY(w)$
  Apply Encoder $\leftarrow E(\phi)$
  Apply Hadamard Gate to a . Begin Swap Test
  for t = 1 to k **do**
    Apply CSWAP(a, $n-k+t$, $n+t$)
  end for
  Apply Hadamard Gate to a . End Swap Test
  $\lambda \leftarrow$ Measure(a)
  Loss ($\lambda\ s$)
  Update ϕ
end for
return ϕ

3) EIP Cell Training Procedure.: LEXIQL's encoder contains parameters ϕ are not dependent on the processors' parameters θ. Therefore, LEXIQL's circuit is designed around the encoder, such that the processor modules must incorporate the behavior of the encoder, which is trained before the processor.

LEXIQL's encoder is derived from a quantum autoencoder, which is delineated into subsystems A and B, where A and B correspond to $n-k$ and k qubits, respectively. Recall that the objective of the encoder is to compress n qubits of data to $n-k$ qubits, leaving k qubits empty. Training the autoencoder requires additional qubits in the form of a "reference space", denoted as spanning k qubits. The optimization objective is to maximize the expected fidelity F between the input of the autoencoder ($|\ \ _i\rangle_{\text{AB}}$ and its output p_i^{out}) [52].

$$\max \text{F}(|\ \ _i\rangle_{\text{AB}}\ \ p_i^{\text{out}}) \tag{4}$$

LEXIQL maximizes fidelity by employing the Swap Test, a quantum algorithm used to determine the similarity between quantum states [6], acting over the encoder circuit and the reference space [6]. This objective can be reduced to maximizing the probability of observing any arbitrary pure fixed state s; see [52] for a full proof of this equivalence. For simplicity, LEXIQL maximizes the fidelity with respect to $|0\rangle$ state on the auxiliary qubits. We provide the algorithm for the training procedure of LEXIQL's encoder in Algorithm 1.

Following the training of the quantum autoencoder, the processor modules are trained independently of the encoder, with the sole objective of minimizing classification loss. The processor modules are placed alongside the encoder and data injectors to train when the EIP cells are assembled.

We provide the algorithm for training the processor modules in Algorithm 2. The training process for the processors is

Authorized licensed use limited to: Universita degli Studi di Salerno. Downloaded on July 02,2026 at 10:13:19 UTC from IEEE Xplore. Restrictions apply.

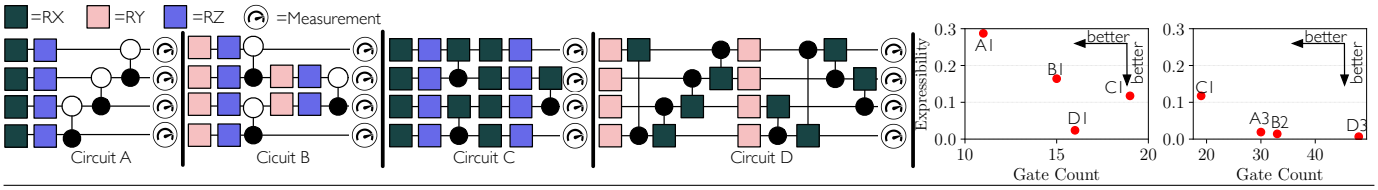


Fig. 6. *Circuits [A, B, C] are used in* LEXIQL. *Circuits [A, B, C, D] correspond to circuits [2, 11, 7, 14] from [58], respectively, in increasing order of expressibility.* LEXIQL *assembles an ensemble consisting of five members from Circuit A with depth 3 (A3), five members from Circuit B (B2), and five members from Circuit C (C1). Circuit D is excluded due to the finding that it is heavily impacted by noise (shown in Sec. V). The expressibility and gate counts for [A, B, C, D] are shown as well. While D3 has the lowest and, hence, best expressibility, it has a significant gate count, making it highly error-prone.*

Algorithm 2 Training Procedure of LEXIQL's Processor

Require: Documents D, Classification Labels Y, Number of iterations N, Trained Encoder $E(\phi)$, Processor $P(\theta_i)$

for i = 1 to N **do**
  Generate batch of data and labels $x_i \ y_i \leftarrow D \ Y$
  for w in x_i **do**
    if w is first word in x_i **then**
      Inject first word $\leftarrow RX(w) \ RY(w)$
    else
      Apply EIP cell $\leftarrow E(\phi) \ I(w) \ P(\theta_i)$
    end if
    $y_i \leftarrow$ Measure
    Loss $\leftarrow$ BinaryCrossEntropyLoss$(y_i \ y_i)$
    Update θ_i
  end for
  $\theta^* \leftarrow \arg\min_{p_i \in P} Loss(\theta)$
end for
return θ^*

simplified by keeping the encoder parameters constant during training. Additionally, limiting the tunable parameters in the training of the processors stabilizes training by limiting problems arising from vanishing gradients. Vanishing gradients occur when gradients approach zero, leading to difficult or impossible-to-train circuits, and are a well-known challenge in scaling quantum machine learning circuits [70]. LEXIQL's approach to encoder training, conducted independently from the processor modules, simplifies the gradient complexities during processor training, effectively addressing the challenge of vanishing gradients.

While addressing vanishing gradients can enhance the resilience of training processes, the presence of uncontrolled NISQ-era noise can still impair the accuracy and dependability of text classification. Driven by the goal to improve resistance to the impacts of noise, LEXIQL employs an ensemble strategy that incorporates variety through the use of multiple distinct quantum members.

D. Key Idea 3: LEXIQL*'s Ensemble of Diverse Learners.*

LEXIQL employs an ensemble approach, combining multiple distinct quantum circuits to enhance classification accuracy in environments with quantum noise. Ensembles leverage diversity to improve accuracy, resilience against noise, and resistance to individual circuits' blind spots in classifying certain cases. To this end, LEXIQL constructs a diverse ensemble of circuits – Circuits [A, B, C] from Figure 6.

An alternative is a homogeneous ensemble with high expressibility – high expressibility circuits are likely to improve classification accuracy [58]. Expressibility gauges a circuit's ability to generate states that effectively represent the Hilbert Space [58]. The expressibility metric is calculated by comparing the distribution of states obtained by sampling parameters of the Variational Quantum Circuit (VQC) to the uniform Haar-random distribution. The uniformity of the generated states indicates the expressibility of a circuit, where sufficient expressive power is crucial for obtaining near-optimal solutions [42]. The number of quantum layer repetitions, known as depth, tends to increase expressibility for ansatzes until a point of saturation [58].

Constructing an ensemble from a homogeneous set of circuits with highly expressible designs may appear advantageous due to their depth and numerous tunable parameters. Specifically, Circuit D from Figure 6 with a depth of three (D3) would appear to be a reasonable strategy to construct an ensemble. However, such an approach has limitations as a uniform ensemble is vulnerable to correlated failures from training and noise, and deeper circuits are inherently more affected by NISQ-era noise, offsetting their expressibility advantage – as confirmed in our evaluation of homogeneous D3 ensemble (Sec. V). To mitigate this challenge, LEXIQL adopts a strategy of assembling a diverse ensemble of circuits.

LEXIQL constructs a diverse ensemble based on expressibility. However, given the high error rates on NISQ-era hardware, depth can come at the expense of overall fidelity. We show an example of this noise-expressibility tradeoff in Figure 7. The figure demonstrates that a deep, highly expressible circuit is more adversely impacted by the noise on quantum hardware than a shallow circuit with lower expressibility.

LEXIQL employs a novel ensemble of circuits [A, B, C] from Figure 6 with depths to match their expressibility levels – finding a balance between expressibility and error-proneness due to higher depth. Circuit A, being the least expressible, is assigned a depth of three to enhance its expressibility, denoted as A3 for simplicity. Circuit B, with moderate expressibility, is set to a depth of two. The most expressible circuit, Circuit C, operates at a depth of one. Circuit D is omitted due to its high noise sensitivity caused by the extensive need for SWAP operations, which we later demonstrate in the evaluation (Sec. V). Consequently, LEXIQL includes five A3 circuits, five

Authorized licensed use limited to: Universita degli Studi di Salerno. Downloaded on July 02,2026 at 10:13:19 UTC from IEEE Xplore. Restrictions apply.

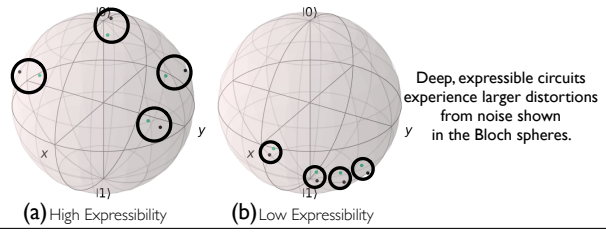


Fig. 7. *Randomly selected parameters for a deep, expressive circuit (a) and a less deep, less expressive circuit (b). Noise is introduced to the systems as shown by shifts from darker points to lighter points. (a) exhibits larger distortions than (b) as noise has a lesser impact on low-expressibility circuits.*

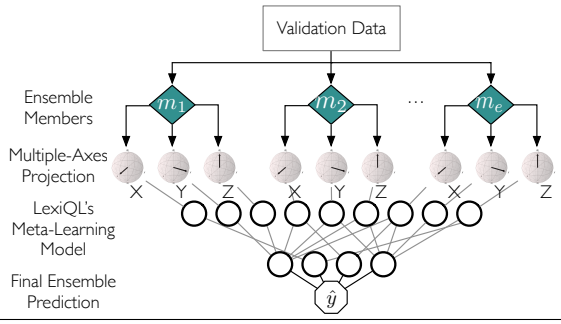


Fig. 8. LEXIQL*'s ensemble stacking technique leverages multiple-axes projection across the X, Y, and Z basis to improve classification.*

B2 circuits, and five C1 circuits, creating a diverse ensemble that balances the expressibility and noise vulnerability.

1) Ensemble Decision Procedure.: An open question with ensemble techniques is determining an optimal method for aggregating multiple predictions from ensemble members into a single effective prediction y. One simple yet effective approach is the majority rules decision procedure, where each of the m members contributes an equal vote [5]. However, this simple decision procedure is often sub-optimal, not making the best use of the more capable ensemble members.

A more intricate procedure, stacking, is defined by the use of a meta-learning model to aggregate ensemble predictions [67]. Stacking enables the learning of connections between ensemble members, leading to a more refined prediction [27], [14]. For instance, the meta-learning model might recognize cases where more trainable, less expressible models should be used in place of more expressible, less trainable models by inadvertently learning which models may have suffered from vanishing gradients for a particular dataset. We find in our evaluation (Sec. V) that the use of a separate meta-learning model brings an improvement to the majority rules decision procedure. However, it is not quantum-aware and, therefore, is limited in the domain of quantum ensemble members. LEXIQL, therefore, proposes a quantum-specific approach to ensemble stacking to strengthen the predictive capability of the quantum ensembles that it employs.

2) LEXIQL*'s Multiple-Axes Projection.:* Recall that quantum states collapse to classical states upon measurement, losing information that had been stored in the higher-dimensional Hilbert Space. LEXIQL, therefore, introduces measurement in multiple quantum axes to extract additional information from quantum circuit measurement compared to measurement on a single axis. Specifically, in three separate circuit runs, the state of qubit 0 is projected to each (X, Y, and Z) axis before measurement. All measurements are then passed as inputs to a meta-learning model, providing additional context from each ensemble member. This approach, illustrated in Figure 8, aims to address the limitation of representing complex information within Hilbert Space when reduced to a single probability. At first glance, it may appear this introduces complexity to the training of the quantum circuits. However, in our evaluation (Sec. V), we show that notable improvement is observed when using this approach without any additional training overhead.

E. Putting It All Together.

Once LEXIQL's training for each ensemble member is completed, all members are evaluated to obtain their outputs using multiple-axes projection. Capturing the training outputs is required to train the meta-learning model. This procedure is then repeated for each quantum machine noise model used in the evaluation. LEXIQL's meta-learning model is trained in accordance with quantum machine noise models to be better equipped to evaluate outputs from noisy environments. Inference is performed after training by simulating LEXIQL's circuits using a specified noise model, followed by aggregating the outputs of these models and passing them to the meta-learning model, which provides the final prediction for each data sample.

IV. METHODOLOGY

Datasets. LEXIQL and the competing methods tested are evaluated on five datasets. The first three datasets have been widely used by classical NLP researchers to demonstrate the effectiveness of sentiment analysis using classical NLP methods [16], [66]. The second two datasets are relatively smaller and datasets used for quantum NLP tasks [2], [26].

These first three datasets include Yelp, IMDB, and Amazon – introduced in [28]. These datasets comprise 1000 samples of manually labeled data. Yelp consists of restaurant reviews labeled as positive or negative. IMDB includes labeled movie reviews from [36], sourced from `imdb.com`. Amazon contains reviews on cell phones and accessories, originally sampled in [38], using a 5-point scale where ratings of 1 and 2 are considered negative, and 4 and 5 are positive.

The two other datasets are LambeqMC and LambeqRP, which were created as described in [35]. These synthetic datasets feature sentences with a relatively smaller number of word counts, vocabulary, and number of samples compared to the first three datasets. Due to the widespread usage and recognition of Lambeq datasets, we have chosen them as benchmark datasets for comparative analysis, but we recognize that they are relatively much simpler and, hence, easier to demonstrate effectiveness.

We employ a 70:20:10 training:validation:test split for the datasets. We randomly select samples from each original

Authorized licensed use limited to: Universita degli Studi di Salerno. Downloaded on July 02,2026 at 10:13:19 UTC from IEEE Xplore. Restrictions apply.

dataset with replacement and divide them according to the dataset split ratio to ensure uniformity between training, validation, and testing data. Given the limited training samples in the Lambeq datasets, only a training:test split is used.

Environmental Framework. In the environmental framework of LEXIQL, we utilize Pennylane version 0.28.0 [3] with Python version 3.10.8. Language-specific preprocessing is carried out using a combination of the nltk [4] and scikit-learn frameworks. The management of LEXIQL's Pennylane circuits and their optimizations are built on top of PyTorch [47]. For training, LEXIQL is trained using AWS, with Fargate managing ARM-based machines containing up to 16GB of RAM and 2 virtual cores per machine. Quantum simulations use up-to-date IBM noise models for the machines IBM Cairo, IBM Hanoi, and IBM Kolkata using a Qiskit [1] backend in conjunction with trained Pennylane models. This selection of models was chosen to capture multiple error rates. IBM Kolkata and IBM Cairo maintain similar median error rates ($\approx$ 0 1) for CNOT gates, meaning 10% of all CNOT gates will not execute with the intended behavior. This is important as CNOT gates are key entanglement gates used in LEXIQL, with each error contributing to decreasing fidelity. However, they have high-error gates distributed differently. As of writing, IBM Cairo has high error rates spread throughout the hardware, making high error CNOT gates difficult to avoid, while IBM Kolkata has high error rates concentrated in one region. IBM Hanoi has an error rate for CNOT gates an order of magnitude lower than IBM Kolkata and Cairo.

Preprocessing and Training Details. LEXIQL uses Global Vectors (GloVe), a vector representation for words trained on co-occurrence statistics [49]. GloVe vectors offer a more condensed word representation for quantum circuits than tokenized methods, which, even in smaller models, involve tens of thousands of unique tokens [53]. Specifically, LEXIQL employs Twitter-25 GloVe vectors, transforming all input words into length-25 vectors trained on the co-occurrence of words in Twitter posts. After applying the GloVe representation to text in each dataset, principal component analysis (PCA) is applied to reduce the size of the GloVe vectors to 8 features per word. The 8 PCA features are used to encode an initial word on the uninitialized circuit, while 4 features are used for each subsequent word, with 2 features being encoded on each qubit in the injection site. Subsequently, the vectorized data is normalized. All preprocessing is performed once and then is cached to limit training overhead.

Recall that LEXIQL uses an ensemble approach for classification; accordingly, we note that the training of individual ensemble members is independent. There are three unique ansatzes used in LEXIQL consisting of A3, B2, and C1. Recall from Sec. III that A3 refers to Circuit A with the depth of three layers, and C1 refers to Circuit C with the depth of one layer. Each ansatz has five identical members (each consisting of seven qubit circuits, two injection site qubits, and five compressed state qubits) – therefore, a total of fifteen ensemble members. PyTorch's RMSProp optimizer is employed with a learning rate of 0 01, which we found to work well for LEXIQL's circuits, and training utilizes BinaryCrossEntropy loss. The loss function is applied between the probability measured between the Z-axis of the first qubit of each ensemble member and the labels. Each ensemble member undergoes training for 200 iterations using a batch size of 50, and the best model concerning a withheld validation set is used in the final aggregate model for testing. Training times depend largely on the circuit chosen and the dataset used. All training runtimes are between 220 and 884 minutes using the previously mentioned hardware configuration, with the IMDB dataset having the largest runtimes amongst datasets and Circuit B having the largest runtime amongst circuits. Following training, the aggregate ensemble model is then evaluated using a separate test set, following a widely used practice [29], [33], [68].

Framework for Competing Techniques. We compare LEXIQL against two relevant quantum techniques for evaluation to cover quantum classification cases of deep-learning inspired frameworks (AMPMEAN) and category theory-based frameworks (DISCOCAT). The first quantum technique, referred to as AMPMEAN, is designed to model the amplitude encoding design in [2]. Our AMPMEAN training is performed by first reducing the word vectors by averaging them as specified in the original work, applied to a seven-qubit circuit. Our implementation of AMPMEAN uses a batch size of 50, with 200 iterations and a learning rate of 0 01 with BinaryCrossEntropy loss, maintaining consistent hyperparameters with LEXIQL.

The second quantum technique examined in this work is the Lambeq implementation of DisCoCat, which we refer to as DISCOCAT. DISCOCAT uses the RemoveCups rewriter to improve compatibility with the tested datasets alongside the Lambeq PyTorchTrainer for training. The hyperparameters are the same as those mentioned in the Lambeq documentation and include 100 iterations with a batch size of 10 and a learning rate of 0 01. Recall that the strict syntax requirements for DISCOCAT restrict compatibility with many real-world datasets. We found this to be the case for the Yelp dataset, where we had to restrict the model size to two words per document, as the use of more words broke compatibility.

The classical technique we examine in this work is a transformer model, originally proposed by [62]. Our implementation uses the Pytorch Transformer model and tokens pretrained by Bert [12] with a vocabulary size of 30,000. We use CrossEntropyLoss for the loss function and AdamOptimizer as the optimizer with a learning rate of 001 with a batch size of 50. In our evaluation, we examine three different-sized transformer models, which we construct based on the number of parameters relative to LEXIQL's model, spanning 5×, 50×, and 500× parameter sizes.

Figures of Merit. To capture the effectiveness of text classification, our evaluation figure of merit is accuracy. Accuracy evaluates the predictive capability of models, which we employ to compare different techniques in the evaluation.

Authorized licensed use limited to: Universita degli Studi di Salerno. Downloaded on July 02,2026 at 10:13:19 UTC from IEEE Xplore. Restrictions apply.

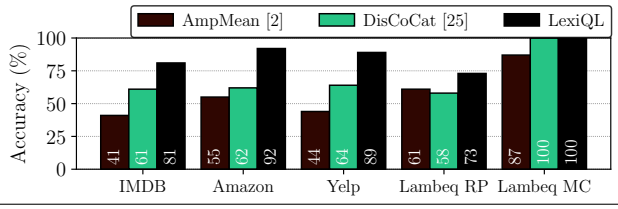


Fig. 9. LEXIQL *achieves significantly better accuracy compared to prior methods across different datasets.*

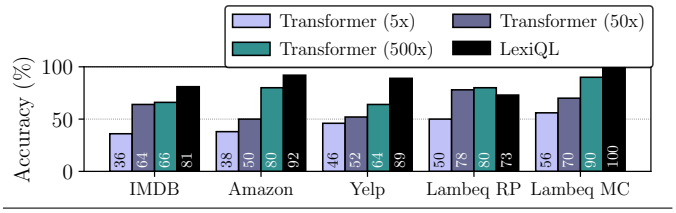


Fig. 10. LEXIQL *performs competitively when compared to classical models with up to a 500-fold increase in the number of parameters.*

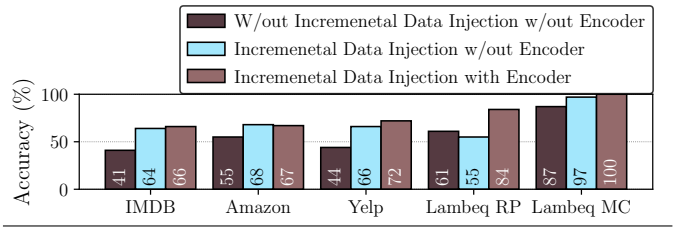


Fig. 11. *Impact of* LEXIQL*'s incremental data injection and its encoder on accuracy are significant (*LEXIQL*'s ensemble technique is not employed to understand the impact of incremental data injection and encoder in isolation).*

V. EVALUATION AND ANALYSIS

This section assesses the effectiveness of LEXIQL and provides an analysis of its key design elements.

LEXIQL's Key Results. We first highlight that LEXIQL provides significant improvements in accuracy compared to competing quantum methods. In fact, Figure 9 shows an average improvement in accuracy of 29% and 18% compared to AMPMEAN and DISCOCAT, respectively. We attribute LEXIQL's large accuracy improvement over AMPMEAN to the limited scalability of data averaging used in AMPMEAN, where larger sequences of up to 15 words are averaged together to a single vector.

Recall that the evaluated datasets include small synthetic Lambeq datasets (LambeqMC and LambeqRP), characterized by short sentences, and the more complex IMDB, Amazon, and Yelp datasets that capture more realistic language usage. LambeqMC maintains the smallest vocabulary among datasets and, hence, multiple models are able to achieve 100% accuracy on the dataset with fast convergence times. For the LambeqMC dataset, LEXIQL achieves 90% accuracy at epoch 10 and 100% accuracy at epoch 17.

While DISCOCAT performs more similarly with LEXIQL on synthetic datasets, its scalability diminishes on more challenging datasets compared to LEXIQL. Recall, DISCOCAT requires the use of a syntax parser, which is not compatible with many words and phrases, limiting the applicability of DISCOCAT with more realistic datasets [60]. Particularly for the Amazon dataset, LEXIQL showcases a 30% accuracy improvement over DISCOCAT, contributing to an overall average accuracy improvement of 27% when examining realistic datasets (IMDB, Amazon, Yelp).

We note that it is challenging to provide a fair comparison with classical methods. This is why, typically, prior quantum papers have refrained from such comparisons, and there is increasing consensus that some quantum areas should be explored without excessive focus on speedup, for now [8]. While classical methods can be scaled to more parameters, the relatively limited size of these datasets renders them less conducive to large classical deep-learning models. The quantum circuits tend to demonstrate a more efficient generalization to smaller training datasets compared to their classical counterparts [7] – which contributes toward LEXIQL's effectiveness. Nevertheless, we cautiously note that despite our encouraging empirical results, we are not claiming or hinting toward a demonstrated quantum advantage for QNLP tasks.

As Figure 10 shows, in our experiments, we found that LEXIQL exhibits competitive performance when benchmarked against certain transformer-based classical models, even with a higher number of parameters. Notably, LEXIQL achieves higher accuracy for all three realistic datasets (IMDB, Amazon, Yelp) when compared to the classical models tested (Figure 10).

To further explore why LEXIQL is so effective, we discuss next how the key components of LEXIQL contribute to its effectiveness on the tested datasets.

Why does LEXIQL work effectively? LEXIQL*'s Encoder-Inject-Processor (EIP) cell modules increase* LEXIQL*'s effectiveness.* In enhancing the accuracy of quantum computing methods in processing complex datasets, LEXIQL emerges as a frontrunner, outperforming both competing quantum approaches and similar classical models. We explore the impact of LEXIQL's mechanisms, which contribute to its performance. Recall that LEXIQL employs incremental data injection to upload data throughout its circuits. Additionally, LEXIQL's encoder repeatedly compresses the state of the quantum circuit to free up qubits for incremental data injection, which mitigates state-data interference. To better demonstrate the effectiveness of these mechanisms, Figure 11 shows how accuracy changes when each of these mechanisms is excluded from LEXIQL. We observe that each design element tested (incremental data injection and the encoder) allows LEXIQL to achieve higher accuracies. Particularly with the challenging IMDB dataset, these mechanisms together yield a 30% increase in accuracy. Note that, for easier interpretation and effect isolation, Figure 11 does not include LEXIQL's ensemble technique or its effects – which are analyzed next.

LEXIQL*'s ensemble technique increases* LEXIQL*'s effectiveness.* The effectiveness of different ensemble techniques is shown in Figure 12. Recall that collapsing quantum states

Authorized licensed use limited to: Universita degli Studi di Salerno. Downloaded on July 02,2026 at 10:13:19 UTC from IEEE Xplore. Restrictions apply.

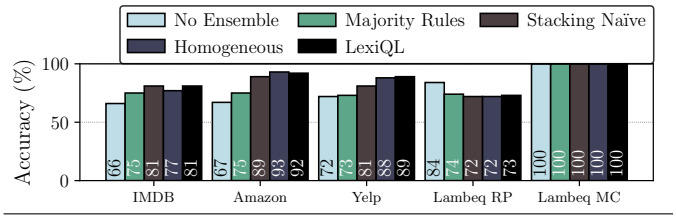


Fig. 12. LEXIQL*'s multiple-axes projection technique achieves the highest accuracy when compared to other ensemble aggregation methods.*

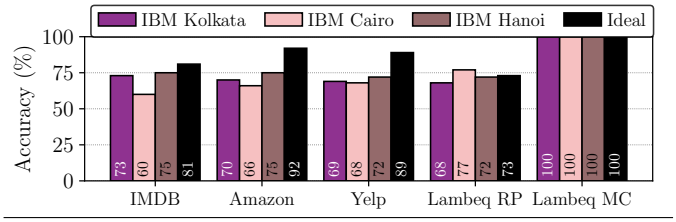


Fig. 13. LEXIQL *achieves high accuracy even under noisy conditions presented by NISQ-era systems (IBM Kolkata, IBM Cairo, and IBM Hanoi).*

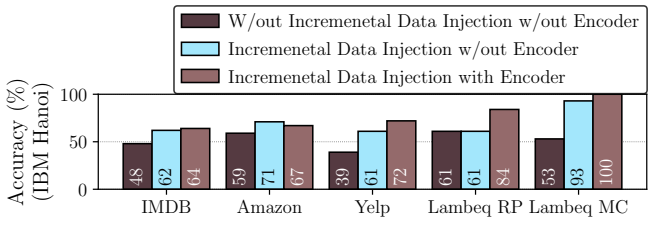


Fig. 14. *Impact of* LEXIQL*'s incremental data injection and encoder under noisy environment (IBM Hanoi).*

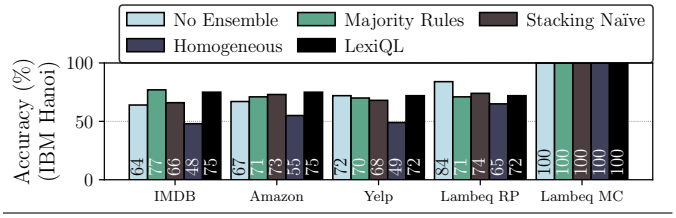


Fig. 15. LEXIQL*'s diverse ensemble technique outperforms particularly under noise compared to homogeneous ensemble technique (IBM Hanoi).*

through measurement in a single projection takes information from higher-dimensional Hilbert Space to a single probability, losing information stored in the quantum system. Therefore, it is critical to ensure that additional projections are measured and provide additional information from the quantum space. LEXIQL's multiple-axes projection leverages a meta-learning model to learn relationships between multiple projections over all ensemble members to improve accuracy. Consequently, we observe that LEXIQL's multiple-axes projection achieves the highest accuracy for all ensemble methods tested. Multiple-axes projection achieves up to 8% improvement in accuracy over naïve stacking.

Figure 12 also shows how LEXIQL performs with a deep and expressive, homogeneous ensemble compared to its diverse ensemble of varying depths and expressibility. We note there is variance among ensemble members for the LameqRP dataset due to few training samples, which creates overfitting. The overfitting becomes more pronounced with more members; each overfits to different parts of the training set, and hence, the ensemble can create a non-desired side-effect for a very small synthetic dataset. We observe that although it appears that a homogeneous ensemble performs competitively in noiseless simulation, and hence, should be sufficient, we show that LEXIQL's diverse ensemble is more noise-resistant, and, therefore, its effectiveness is amplified in noisy conditions, as discussed next.

LEXIQL on NISQ Quantum Machines. We explore superconducting-qubit noise by evaluating the effects of noise from quantum machines IBM Kolkata, IBM Cairo, and IBM Hanoi. It is important to note that quantum computer hardware technology is developing rapidly, and thus, machines can introduce vastly different amounts of noise. Recall that IBM Kolkata and IBM Cairo have CNOT error rates an order of magnitude higher than IBM Hanoi. Therefore, noise has a larger impact on these machines. We observe the effects of this in Figure 13, where LEXIQL's accuracy is most similar to ideal (noiseless) when evaluated using IBM Hanoi's noise model. Despite high noise in IBM Kolkata and IBM Cairo, LEXIQL continues to perform competitively. In the following discussion, we examine the effectiveness of LEXIQL's components when noise is present, using IBM Hanoi's noise model. We note that the average gate count for LexiQL's circuits is 339 gates (compared to AmpMean's 285 gates and DisCoCat's 53 gates). Despite the larger gate count in LexiQL, it performs better in noisy environments because of its effective noise mitigation strategies.

Figure 14 shows the effect of LEXIQL's incremental data injection and LEXIQL's encoder in noisy simulation using IBM Hanoi's noise model. First, we observe that incremental data injection continues to improve accuracy in noisy simulation, achieving an average increase in accuracy of 12 4% for all datasets tested. Additionally, we find that despite noise impacting the fidelity of the encoder, the encoder manages to mitigate state-data interference, contributing to an average improvement of 7 8%, demonstrating the impact of the encoder in a noisy quantum environment.

Next, Figure 15 shows the effectiveness of multiple ensemble approaches in a noisy environment (IBM Hanoi). We observe that LEXIQL's multiple-axes projection continues to outperform other ensemble techniques even in noisy environments. As multiple-axes projection measures in multiple axes, it adds redundancy to perturbations from noise compared to the standard approach of using measurement in a single axis.

Additionally, recall that LEXIQL's ensemble is constructed using three unique ansatzes, each comprising five ensemble members. We further explore the impact of noise on this approach when compared to a reasonable homogeneous ensemble. We find that the homogeneous ensemble is not robust to noise, as each ensemble member has similar vulnerabilities. This is evident as the homogeneous ensemble achieves

Authorized licensed use limited to: Universita degli Studi di Salerno. Downloaded on July 02,2026 at 10:13:19 UTC from IEEE Xplore. Restrictions apply.

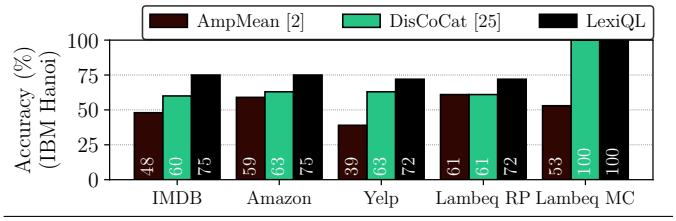


Fig. 16. AMPMEAN, DISCOCAT, *and* LEXIQL*'s accuracy under noisy condition (IBM Hanoi).* LEXIQL *competitively outperforms prior methods.*

around 50% accuracy for complex datasets, being unable to mitigate noise. In contrast, LEXIQL's diverse ensemble members strengthen resiliency to noise and outperform the homogeneous ensemble in each noisy case, achieving near 75% accuracy for realistic datasets.

Finally, we show in Figure 16 that LEXIQL provides higher accuracy across different datasets when compared to competing methods (AMPMEAN and DISCOCAT) under real-system noise (IBM Hanoi). LEXIQL outperforms AMPMEAN and DISCOCAT by 26 6% accuracy and 9 4% accuracy on average respectively over all five tested datasets. Most notably, LEXIQL outperforms AMPMEAN and DISCOCAT on the complex datasets – pushing the state of the art in quantum NLP and paving the way for other further research. Our results demonstrate LEXIQL can encode and train realistic datasets (e.g., IMDB and Amazon reviews with 15 word-long sentences) with reasonable depth while effectively mitigating noise. As quantum hardware technology improves (higher decoherence time and lower noise), we anticipate that LEXIQL's benefits will further increase.

VI. RELATED WORK

Recent research has shown that Quantum Machine Learning (QML) offers significant promise, with the potential of outperforming traditional machine learning capabilities. QML has been shown to generalize better with fewer training data when compared to classical counterparts [7] and has the potential to rival classical counterparts in many aspects [54]. QML has been demonstrated in the NISQ era for a variety of applications including image classification, image similarity detection, and image generation [48], [57], [51], [56], [55]. A framework leveraging an ensemble of diverse quantum classifiers for image classification is proposed in [57]. Following image classification, the work in [56] expands quantum image processing to the unsupervised learning task of clustering-based similarity detection within images. Quantum image processing is further explored in [55], where a design for quantum GAN (generative adversarial network) is introduced, providing image generation by leveraging multiple quantum circuits together to generate images. While many develops in QML have been made for image processing, the potential of QML for NLP has not been explored in depth, especially for NISQ-era quantum computing.

One research direction in QNLP involves quantum-enhanced NLP, where existing classical NLP algorithms are augmented with quantum layers [18]. These algorithms often adopt a transfer-learning-based technique known as "dressing", where quantum layers are interposed between classical neural networks on both ends [37]. However, when employing large classical networks in this manner for QNLP, the computational burden is primarily handled by the classical networks [59]. In [59], classical LSTM layers are replaced with quantum-dressed layers for a task involving labeling the figure of speech of a word (determinant, noun, verb). While the same work extends this dressing approach to a transformer, the authors note impractical training times exceeding 100 hours for a single epoch. This is in contrast to LEXIQL, which is able to train in parallel, completing training in around 12 hours. Another attempt at creating a dressed quantum LSTM is presented in [60], which demonstrates no advantage over the classical case on a synthetic dataset. Notably, this work also explores Distributional Compositional Categorical (DisCoCat) models in the QNLP context.

In the work [60], practical challenges with DisCoCat-based models are identified. One practical challenge includes difficulties in simulating the models due to high computational complexity. Additionally, the manuscript also highlights hurdles faced in finding real-world datasets that align with DiscCoCat's parsing technique yet remain small enough for current quantum circuit simulators. This is in contrast to LEXIQL, which has demonstrated proficiency with different real-world datasets without the need for intricate grammatical parsing. In contrast, the work by Kartsaklis et al. [26] relies on a synthetic dataset (LambeqMC), to achieve high accuracy in sentiment analysis. However, the scalability of such approaches to more complex datasets is limited – LEXIQL's techniques eliminate these bottlenecks and show for the first time that it is possible to achieve high accuracy on complex datasets for quantum NLP tasks for NISQ-era quantum machines.

VII. CONCLUSION

In this work, we present LEXIQL, a technique for the NLP task of text classification using quantum computing. We demonstrate the performance of LEXIQL on five datasets of varying difficulty and demonstrate the wide range of domain applicability for LEXIQL. We also demonstrate that LEXIQL can achieve high accuracy in noiseless simulation and can also effectively mitigate NISQ-era noise. LEXIQL is available as an open-source framework for quantum text classification at https://zenodo.org/records/10980744.

ACKNOWLEDGMENT

We thank the reviewers for their constructive feedback. This work was supported by NSF Awards (2144540, 2124897, and 1910601), MathWorks Faculty Award, Northeastern University, and Rice University. This research used resources of the Oak Ridge Leadership Computing Facility, which is a DOE Office of Science User Facility supported under Contract DE-AC05-00OR22725. We utilized ChatGPT, an AI language model developed by OpenAI, for partial assistance in drafting the text, and all generated content was thoroughly reviewed and edited for accuracy.

Authorized licensed use limited to: Universita degli Studi di Salerno. Downloaded on July 02,2026 at 10:13:19 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] Gadi Aleksandrowicz, Thomas Alexander, Panagiotis Barkoutsos, Luciano Bello, Yael Ben-Haim, David Bucher, F Jose Cabrera-Hernández, Jorge Carballo-Franquis, Adrian Chen, Chun-Fu Chen, et al. Qiskit: An open-source framework for quantum computing. *Accessed on: Mar*, 16, 2019.

[2] Aaranya Alexander and Dominic Widdows. Quantum text encoding for classification tasks. In *2022 IEEE/ACM 7th Symposium on Edge Computing (SEC)*, pages 355–361. IEEE, 2022.

[3] Ville Bergholm, Josh Izaac, Maria Schuld, Christian Gogolin, M Sohaib Alam, Shahnawaz Ahmed, Juan Miguel Arrazola, Carsten Blank, Alain Delgado, Soran Jahangiri, et al. Pennylane: Automatic differentiation of hybrid quantum-classical computations. *arXiv preprint arXiv:1811.04968*, 2018.

[4] Steven Bird, Ewan Klein, and Edward Loper. *Natural language processing with Python: analyzing text with the natural language toolkit*. " O'Reilly Media, Inc.", 2009.

[5] Leo Breiman. Bagging predictors. *Machine learning*, 24:123–140, 1996.

[6] Harry Buhrman, Richard Cleve, John Watrous, and Ronald De Wolf. Quantum fingerprinting. *Physical Review Letters*, 87(16):167902, 2001.

[7] Matthias C. Caro, Hsin-Yuan Huang, M. Cerezo, Kunal Sharma, Andrew Sornborger, Lukasz Cincio, and Patrick J. Coles. Generalization in quantum machine learning from few training data. *Nature Communications*, 13(1), August 2022.

[8] Davide Castelvecchi. The ai-quantum computing mash-up: will it revolutionize science? *Nature*, 2024.

[9] Samuel Yen-Chi Chen, Chao-Han Huck Yang, Jun Qi, Pin-Yu Chen, Xiaoli Ma, and Hsi-Sheng Goan. Variational quantum circuits for deep reinforcement learning. *IEEE Access*, 8:141007–141024, 2020.

[10] Bob Coecke, Mehrnoosh Sadrzadeh, and Stephen Clark. Mathematical foundations for a compositional distributional model of meaning, 2010.

[11] Samudra Dasgupta and Travis S Humble. Characterizing the stability of nisq devices. In *2020 IEEE International Conference on Quantum Computing and Engineering (QCE)*, pages 419–429. IEEE, 2020.

[12] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.

[13] Xianzhong Ding, Le Chen, Murali Emani, Chunhua Liao, Pei-Hung Lin, Tristan Vanderbruggen, Zhen Xie, Alberto Cerpa, and Wan Du. Hpc-gpt: Integrating large language model for high-performance computing. In *Proceedings of the SC'23 Workshops of The International Conference on High Performance Computing, Network, Storage, and Analysis*, pages 951–960, 2023.

[14] Federico Divina, Aude Gilson, Francisco Goméz-Vela, Miguel García Torres, and José F Torres. Stacking ensemble learning for short-term electricity consumption forecasting. *Energies*, 11(4):949, 2018.

[15] Yuxuan Du, Tao Huang, Shan You, Min-Hsiu Hsieh, and Dacheng Tao. Quantum circuit architecture search for variational quantum algorithms. *npj Quantum Information*, 8(1):62, 2022.

[16] Mateus Espadoto, Rafael M Martins, Andreas Kerren, Nina ST Hirata, and Alexandru C Telea. Toward a quantitative survey of dimension reduction techniques. *IEEE transactions on visualization and computer graphics*, 27(3):2153–2173, 2019.

[17] Edward Grefenstette and Mehrnoosh Sadrzadeh. Concrete models and empirical evaluations for the categorical compositional distributional model of meaning. *Computational Linguistics*, 41(1):71–118, 2015.

[18] Raffaele Guarasci, Giuseppe De Pietro, and Massimo Esposito. Quantum natural language processing: Challenges and opportunities. *Applied Sciences*, 12(11):5651, 2022.

[19] Chang-Jiang Huang, Hailan Ma, Qi Yin, Jun-Feng Tang, Daoyi Dong, Chunlin Chen, Guo-Yong Xiang, Chuan-Feng Li, and Guang-Can Guo. Realization of a quantum autoencoder for lossless compression of quantum data. *Physical Review A*, 102(3):032412, 2020.

[20] He-Liang Huang, Yuxuan Du, Ming Gong, Youwei Zhao, Yulin Wu, Chaoyue Wang, Shaowei Li, Futian Liang, Jin Lin, Yu Xu, et al. Experimental quantum generative adversarial networks for image generation. *Physical Review Applied*, 16(2):024051, 2021.

[21] Eliu A Huerta, Asad Khan, Edward Davis, Colleen Bushell, William D Gropp, Daniel S Katz, Volodymyr Kindratenko, Seid Koric, William TC Kramer, Brendan McGinty, et al. Convergence of artificial intelligence and high performance computing on nsf-supported cyberinfrastructure. *Journal of Big Data*, 7:1–12, 2020.

[22] Naveed Hussain, Hamid Turab Mirza, Ghulam Rasool, Ibrar Hussain, and Mohammad Kaleem. Spam review detection techniques: A systematic literature review. *Applied Sciences*, 9(5):987, 2019.

[23] Sofiene Jerbi, Lukas J Fiderer, Hendrik Poulsen Nautrup, Jonas M Kübler, Hans J Briegel, and Vedran Dunjko. Quantum machine learning beyond kernel methods. *Nature Communications*, 14(1):517, 2023.

[24] Thorsten Joachims. Text categorization with support vector machines: Learning with many relevant features. In *European conference on machine learning*, pages 137–142. Springer, 1998.

[25] Melvin Johnson, Mike Schuster, Quoc V. Le, Maxim Krikun, Yonghui Wu, Zhifeng Chen, Nikhil Thorat, Fernanda Viégas, Martin Wattenberg, Greg Corrado, Macduff Hughes, and Jeffrey Dean. Google's multilingual neural machine translation system: Enabling zero-shot translation, 2017.

[26] Dimitri Kartsaklis, Ian Fan, Richie Yeung, Anna Pearson, Robin Lorenz, Alexis Toumi, Giovanni de Felice, Konstantinos Meichanetzidis, Stephen Clark, and Bob Coecke. lambeq: An Efficient High-Level Python Library for Quantum NLP. *arXiv preprint arXiv:2110.04236*, 2021.

[27] Waqas Khan, Shalika Walker, and Wim Zeiler. Improved solar photovoltaic energy generation forecast using deep learning-based ensemble stacking approach. *Energy*, 240:122812, 2022.

[28] Dimitrios Kotzias, Misha Denil, Nando De Freitas, and Padhraic Smyth. From group to individual labels using deep features. In *Proceedings of the 21th ACM SIGKDD international conference on knowledge discovery and data mining*, pages 597–606, 2015.

[29] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. *Advances in neural information processing systems*, 25, 2012.

[30] David D Lewis, Yiming Yang, Tony Russell-Rose, and Fan Li. Rcv1: A new benchmark collection for text categorization research. *Journal of machine learning research*, 5(Apr):361–397, 2004.

[31] Jiwei Li, Will Monroe, Alan Ritter, Michel Galley, Jianfeng Gao, and Dan Jurafsky. Deep reinforcement learning for dialogue generation. *arXiv preprint arXiv:1606.01541*, 2016.

[32] Junyu Liu, Minzhao Liu, Jin-Peng Liu, Ziyu Ye, Yunfei Wang, Yuri Alexeev, Jens Eisert, and Liang Jiang. Towards provably efficient quantum algorithms for large-scale machine-learning models. *Nature Communications*, 15(1):434, 2024.

[33] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.

[34] Yunchao Liu, Srinivasan Arunachalam, and Kristan Temme. A rigorous and robust quantum speed-up in supervised machine learning. *Nature Physics*, 17(9):1013–1017, 2021.

[35] Robin Lorenz, Anna Pearson, Konstantinos Meichanetzidis, Dimitri Kartsaklis, and Bob Coecke. Qnlp in practice: Running compositional models of meaning on a quantum computer. *Journal of Artificial Intelligence Research*, 76:1305–1342, April 2023.

[36] Andrew Maas, Raymond E Daly, Peter T Pham, Dan Huang, Andrew Y Ng, and Christopher Potts. Learning word vectors for sentiment analysis. In *Proceedings of the 49th annual meeting of the association for computational linguistics: Human language technologies*, pages 142–150, 2011.

[37] Andrea Mari, Thomas R Bromley, Josh Izaac, Maria Schuld, and Nathan Killoran. Transfer learning in hybrid classical-quantum neural networks. *Quantum*, 4:340, 2020.

[38] Julian McAuley and Jure Leskovec. Hidden factors and hidden topics: understanding rating dimensions with review text. In *Proceedings of the 7th ACM conference on Recommender systems*, pages 165–172, 2013.

[39] Konstantinos Meichanetzidis, Stefano Gogioso, Giovanni de Felice, Nicolò Chiappori, Alexis Toumi, and Bob Coecke. Quantum natural language processing on near-term quantum computers. In Benoît Valiron, Shane Mansfield, Pablo Arrighi, and Prakash Panangaden, editors, *Proceedings 17th International Conference on Quantum Physics and Logic, QPL 2020, Paris, France, June 2 - 6, 2020*, volume 340 of *EPTCS*, pages 213–229, 2020.

[40] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*, 2013.

[41] Yasmin Moslem, Rejwanul Haque, John D Kelleher, and Andy Way. Adaptive machine translation with large language models. *arXiv preprint arXiv:2301.13294*, 2023.

[42] Kouhei Nakaji and Naoki Yamamoto. Expressibility of the alternating layered ansatz for quantum computation. *Quantum*, 5:434, 2021.

[43] Andrew Ng et al. Sparse autoencoder. *CS294A Lecture notes*, 72(2011):1–19, 2011.

Authorized licensed use limited to: Universita degli Studi di Salerno. Downloaded on July 02,2026 at 10:13:19 UTC from IEEE Xplore. Restrictions apply.

[44] Michael A Nielsen and Isaac L Chuang. *Quantum computation and quantum information*. Cambridge university press, 2010.

[45] Kamal Nigam, Andrew Kachites McCallum, Sebastian Thrun, and Tom Mitchell. Text classification from labeled and unlabeled documents using em. *Machine learning*, 39:103–134, 2000.

[46] OpenAI, :, Josh Achiam, Steven Adler, Sandhini Agarwal, and others. Gpt-4 technical report, 2023.

[47] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. Pytorch: An imperative style, high-performance deep learning library, 2019.

[48] Tirthak Patel, Daniel Silver, and Devesh Tiwari. Optic: A practical quantum binary classifier for near-term quantum computers. In *2022 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pages 334–339. IEEE, 2022.

[49] Jeffrey Pennington, Richard Socher, and Christopher D Manning. Glove: Global vectors for word representation. In *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*, pages 1532–1543, 2014.

[50] Somayeh Bakhtiari Ramezani, Alexander Sommers, Harish Kumar Manchukonda, Shahram Rahimi, and Amin Amirlatifi. Machine learning algorithms in quantum computing: A survey. In *2020 International joint conference on neural networks (IJCNN)*, pages 1–8. IEEE, 2020.

[51] Aditya Ranjan, Tirthak Patel, Daniel Silver, Harshitta Gandhi, and Devesh Tiwari. Proximl: Building machine learning classifiers for photonic quantum computing. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, pages 834–849, 2024.

[52] Jonathan Romero, Jonathan P Olson, and Alan Aspuru-Guzik. Quantum autoencoders for efficient compression of quantum data. *Quantum Science and Technology*, 2(4):045001, 2017.

[53] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019.

[54] Maria Schuld and Nathan Killoran. Is quantum advantage the right goal for quantum machine learning? *Prx Quantum*, 3(3):030101, 2022.

[55] Daniel Silver, Tirthak Patel, William Cutler, Aditya Ranjan, Harshitta Gandhi, and Devesh Tiwari. Mosaiq: Quantum generative adversarial networks for image generation on nisq computers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 7030–7039, 2023.

[56] Daniel Silver, Tirthak Patel, Aditya Ranjan, Harshitta Gandhi, William Cutler, and Devesh Tiwari. Sliq: quantum image similarity networks on noisy quantum computers. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pages 9846–9854, 2023.

[57] Daniel Silver, Tirthak Patel, and Devesh Tiwari. Quilt: Effective multi-class classification on quantum computers using an ensemble of diverse quantum classifiers. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, pages 8324–8332, 2022.

[58] Sukin Sim, Peter D. Johnson, and Alán Aspuru-Guzik. Expressibility and entangling capability of parameterized quantum circuits for hybrid quantum-classical algorithms. *Advanced Quantum Technologies*, 2(12), October 2019.

[59] Riccardo Di Sipio, Jia-Hong Huang, Samuel Yen-Chi Chen, Stefano Mangini, and Marcel Worring. The dawn of quantum natural language processing, 2021.

[60] Jonas Stein, Ivo Christ, Nicolas Kraus, Maximilian Balthasar Mansky, Robert Müller, and Claudia Linnhoff-Popien. Applying qnlp to sentiment analysis in finance. In *2023 IEEE International Conference on Quantum Computing and Engineering (QCE)*, volume 2, pages 20–25. IEEE, 2023.

[61] Kian Long Tan, Chin Poo Lee, and Kian Ming Lim. A survey of sentiment analysis: Approaches, datasets, and future research. *Applied Sciences*, 13(7):4550, 2023.

[62] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2017.

[63] Ethan Waisberg, Joshua Ong, Mouayad Masalkhi, and Andrew G Lee. Large language model (llm)-driven chatbots for neuro-ophthalmic medical education. *Eye*, pages 1–3, 2023.

[64] Hanrui Wang, Yongshan Ding, Jiaqi Gu, Zirui Li, Yujun Lin, David Z. Pan, Frederic T. Chong, and Song Han. Quantumnas: Noise-adaptive search for robust quantum circuits, 2022.

[65] Hanrui Wang, Jiaqi Gu, Yongshan Ding, Zirui Li, Frederic T. Chong, David Z. Pan, and Song Han. Quantumnat: Quantum noise-aware training with noise injection, quantization and normalization, 2023.

[66] Jason Wei, Maarten Bosma, Vincent Y Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M Dai, and Quoc V Le. Finetuned language models are zero-shot learners. *arXiv preprint arXiv:2109.01652*, 2021.

[67] David H Wolpert. Stacked generalization. *Neural networks*, 5(2):241–259, 1992.

[68] Zhilin Yang, Zihang Dai, Yiming Yang, Jaime Carbonell, Russ R Salakhutdinov, and Quoc V Le. Xlnet: Generalized autoregressive pretraining for language understanding. *Advances in neural information processing systems*, 32, 2019.

[69] Amine Zeguendry, Zahi Jarir, and Mohamed Quafafou. Quantum machine learning: A review and case studies. *Entropy*, 25(2):287, 2023.

[70] Kaining Zhang, Liu Liu, Min-Hsiu Hsieh, and Dacheng Tao. Escaping from the barren plateau via gaussian initializations in deep variational quantum circuits. *Advances in Neural Information Processing Systems*, 35:18612–18627, 2022.

Authorized licensed use limited to: Universita degli Studi di Salerno. Downloaded on July 02,2026 at 10:13:19 UTC from IEEE Xplore. Restrictions apply.

Appendix: Artifact Description/Artifact Evaluation

Artifact Description (AD)

I. Overview of Contributions and Artifacts

A. Paper's Main Contributions

C_1 LEXIQL presents a design and corresponding implementation of a cell-based Quantum Natural Language Processing (QNLP) framework for text classification. LEXIQL introduces a novel incremental data injection approach to process textual data in a quantum circuit. LEXIQL's approach reduces the information loss suffered by prior approaches, which naively rely on averaging all the data upfront in one shot.

C_2 LEXIQL designs EIP (Encode-Inject-Process) cells that compress prior state information, inject new data for learning, and use a processing layer to train for text classification. LEXIQL's EIP cells use a quantum encoder to compress data and free up qubits to inject new data. This architecture enables LEXIQL to train effectively and mitigate the state-data interference challenges created by incremental data injection.

C_3 LEXIQL designs a diverse quantum ensemble to improve classification accuracy. LEXIQL's diverse ensemble approach enhances accuracy and noise resilience by combining multiple quantum models – and by carefully navigating the trade-off between error-proneness (answer fidelity) and expressibility (learning capability) of individual learner models. LEXIQL additionally improves its ensemble by multiple-axes projection. LEXIQL's multiple-axes projection provides significant improvements in accuracy without requiring re-training in different dimensions.

C_4 Our extensive evaluation of five datasets, comprising synthetic and real data, demonstrates the effectiveness of LEXIQL, in both ideal and noisy environments. LEXIQL achieves 87% average accuracy over five datasets in ideal simulation. It is also evaluated under different noise scenarios and achieves a 79% average accuracy under IBM's Hanoi quantum computer's noise environment.

B. Computational Artifacts

A_1 LEXIQL Code: `https://zenodo.org/records/10980744` DOI: `10.5281/zenodo.10980744`

TABLE I
COMPUTATIONAL ARTIFACTS

Artifact ID	Associated Contributions	Associated Figures
A_1	C_1, C_2, C_3, C_4	Figures 9,10,11,12, 13, 14,15,16

II. Artifact Identification

A. Computational Artifact A_1

Relation To Contributions

The artifact presented is associated with the full codebase used to generate the results in LEXIQL along with the associated datasets. The artifact contains the repository for the code and dataset. The datasets are stored in the `datasets` directory. The code specific for generating the plots shown in LEXIQL is located in the `plotting/code` directory. with plots generated to the `plotting/plots` directory. The main entry point to run all code is the `main.py` file.

Expected Results

R_1 The accuracy of LEXIQL on each examined dataset in ideal simulation (IMDB, Amazon, Yelp, LambeqMC, LambeqRP). The bars represented LEXIQL's accuracy are black and labeled as LEXIQL in Figure 9, the accuracy should be higher than competing techniques.

R_2 The accuracy of LEXIQL on each examined dataset with ablation studies to remove individual components for ideal simulation are shown (Figure 11 and in Figure 12). LEXIQL should have higher overall accuracy then each ablation study in ideal simulation.

R_3 The impact of different quantum machine noise models on LEXIQL's accuracy is shown in Figure 13. The accuracy for each noise model should be less than ideal, with IBM Kolkata and IBM Cairo showing lower accuracy than IBM Hanoi.

R_4 LEXIQL should have higher overall accuracy than each ablation in noisy simulation (Figure 14 and Figure 15).

R_5 LEXIQL should have higher accuracy than competing techniques in noisy simulation (Figure 16).

Expected Reproduction Time (in Minutes)

LEXIQL has three distinct steps for execution, consisting of training, circuit evaluation, and meta-learning. Training is performed for each of 15 ensemble members in ideal simulation. Circuit evaluation is performed for each of 15 ensemble members, evaluated in ideal and noisy simulation, for training and testing data. Meta-learning is performed for each noisy environment and the ideal environment, aggregating outputs from all 15 ensemble members. The execution time for each execution step is shown in minutes in Table II. Training execution time may vary slightly between runs, accounting for up to 20% variance.

Artifact Setup (incl. Inputs)

Hardware: The training and circuit evaluation steps are performed using AWS hardware. AWS Fargate managing

Authorized licensed use limited to: Universita degli Studi di Salerno. Downloaded on July 02,2026 at 10:13:19 UTC from IEEE Xplore. Restrictions apply.

TABLE II
COMPUTATIONAL ARTIFACTS

Execution Step	Dataset	Circuit	Time (Minutes)
Training	IMDB	Circuit A	720
Training	IMDB	Circuit B	884
Training	IMDB	Circuit C	528
Training	Amazon	Circuit A	615
Training	Amazon	Circuit B	772
Training	Amazon	Circuit C	486
Training	Yelp	Circuit A	655
Training	Yelp	Circuit B	790
Training	Yelp	Circuit C	494
Training	LambeqMC	Circuit A	282
Training	LambeqMC	Circuit B	350
Training	LambeqMC	Circuit C	216
Training	LambeqRP	Circuit A	289
Training	LambeqRP	Circuit B	367
Training	LambeqRP	Circuit C	220
Circuit Evaluation	IMDB	Circuit A	2
Circuit Evaluation	IMDB	Circuit B	2
Circuit Evaluation	IMDB	Circuit C	1
Circuit Evaluation	Amazon	Circuit A	2
Circuit Evaluation	Amazon	Circuit B	2
Circuit Evaluation	Amazon	Circuit C	1
Circuit Evaluation	Yelp	Circuit A	2
Circuit Evaluation	Yelp	Circuit B	2
Circuit Evaluation	Yelp	Circuit C	1
Circuit Evaluation	LambeqMC	Circuit A	1
Circuit Evaluation	LambeqMC	Circuit B	1
Circuit Evaluation	LambeqMC	Circuit C	1
Circuit Evaluation	LambeqRP	Circuit A	1
Circuit Evaluation	LambeqRP	Circuit B	1
Circuit Evaluation	LambeqRP	Circuit C	1
Meta-Learning	IMDB	All	1
Meta-Learning	Amazon	All	1
Meta-Learning	Yelp	All	1
Meta-Learning	LambeqMC	All	1
Meta-Learning	LambeqRP	All	1

ARM-based machines containing up to 16GB of RAM and 2 virtual cores per machine perform all steps. The meta-learning inference was performed locally using a macOS Ventura 13.5 macbook pro with M2 processor and 16GB RAM.

Software: To implement of LEXIQL, we utilize Pennylane version 0.28.0 with Python version 3.10.8. Language-specific preprocessing is carried out using a combination of the nltk and scikit-learn frameworks. The management of LEXIQL's Pennylane circuits and their optimizations are built on top of PyTorch version 2.1.0. Quantum simulations use up-to-date IBM noise models for the machines IBM Cairo, IBM Hanoi, and IBM Kolkata using a Qiskit backend in conjunction with trained Pennylane models.

Datasets / Inputs: The datasets tested in LEXIQL are included in the `datasets` directory. LEXIQL uses Global Vectors (GloVe), a vector representation for words trained on co-occurrence statistics. After applying the GloVe representation to text in each dataset, principal component analysis (PCA) is applied to reduce the size of the GloVe vectors to 8 features per word. Subsequently, the vectorized data is normalized. These preprocessing steps are cached and stored in the `datasets/cached` directory.

Installation and Deployment: Installation requires installing required packages stored in the included `requirements.txt` file using the command `pip3 -r install requirements.txt` Following installation, running `python3 main.py` with the specified dataset and ensemble member number will begin to train the model. After all ensemble members are trained, circuit evaluation can be performed by running `python3 main.py` with the `--mode=eval` argument. As the meta-learning model requires training data and validation data, this is performed twice, using the `--data_eval_cat=train` and `--data_eval_cat=val` arguments. To simulate each quantum noisy environment, use the `--machine` argument. For example, to simulate the noise for IBM Hanoi, use `--machine=sim_hanoi`. The evaluation data including weights and circuit evaluation outputs will then be saved to the `evaluation_results` directory. Executing the meta-learning model procedure provides the final accuracy measure. This is performed by running `python3 main.py --mode=combine`.

Artifact Execution

The workflow to execute LEXIQL are as follows:

1) Train all 15 ensemble members.
2) Run circuit evaluation for each member, repeated for each noisy environment.
3) Run the meta-learning model to obtain final accuracy numbers, repeating for each noisy environment.
4) Generate the plots used in LEXIQL.

Artifact Analysis (incl. Outputs)

The accuracies generated should by the meta-learning model should match those in Figures 9, 10, 11, 12, 13, 14, 15, and 16 in the paper. Additionally, the accuracies for the ablation experiments, which do not use the meta-learning model, should match the accuracies in Figures 11, 12, 14, and 15.

Authorized licensed use limited to: Universita degli Studi di Salerno. Downloaded on July 02,2026 at 10:13:19 UTC from IEEE Xplore. Restrictions apply.